# Merged Files List


1. tech_interview_preparation.md (3.8 KB)
2. choosing_a_tech_domain.md (3.6 KB)
3. switching_into_tech.md (3.6 KB)
4. linux_administrator.md (3.5 KB)
5. ux_researcher.md (3.5 KB)
6. network_engineer.md (3.5 KB)
7. infrastructure_engineer.md (3.4 KB)
8. tech_resume_guide.md (3.4 KB)
9. sap_consultant_developer.md (3.2 KB)
10. erp_consultant.md (3.2 KB)
11. performance_test_engineer.md (3.1 KB)
12. qa_engineer.md (3 KB)
13. enterprise_application_developer.md (3 KB)
14. salesforce_developer.md (3 KB)
15. iot_developer.md (2.9 KB)
16. systems_software_engineer.md (2.9 KB)
17. sdet.md (2.9 KB)
18. systems_administrator.md (2.8 KB)
19. firmware_engineer.md (2.8 KB)
20. ux_designer.md (2.8 KB)
21. embedded_software_engineer.md (2.8 KB)
22. automation_test_engineer.md (2.8 KB)
23. ui_designer.md (2.8 KB)
24. manual_test_engineer.md (2.7 KB)
25. robotics_software_engineer.md (2.7 KB)
26. product_designer.md (2.6 KB)
27. network_administrator.md (2.5 KB)


# 1. tech_interview_preparation.md

```
# How to Prepare for Tech Interviews (All Experience Levels)

## Role guide
Tech interviews usually run in several rounds, and what they test depends on the role and level. A typical software process has a fi

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: data structures (arrays, linked lists, stacks, queues, hash maps, trees, graphs, heaps) and algorithms (sorting, se
- Programming: be fluent in one language you can code in under pressure.
- Development environment: a plain editor or an online coding practice platform (LeetCode, HackerRank), and a habit of practising ou
- Know the process: ask the recruiter which rounds to expect, whether there is a phone screen, a coding test or a system design roun

### 2. Core and technical skill progression
- Level 1, coding fundamentals: revise data structures and algorithms and solve easier problems for each one before moving to harder
- Level 2, design rounds: low-level design (classes, interfaces and object-oriented design) and high-level system design. In system
- Level 3, behavioral and role-specific rounds: prepare eight to ten stories in the STAR format (Situation, Task, Action, Result) co

### 3. Practical tooling and ecosystem
- Practice: LeetCode, HackerRank and similar platforms, system design courses and books, mock interviews with friends or online.
- Reference: your own notes on patterns and mistakes.
- Behavioral: a written list of STAR stories.

### 4. Hands-on projects and portfolio milestones
- Beginner: solve problems on each core data structure, and explain your solutions out loud.
- Intermediate: do timed mock interviews, practise a low-level design (for example a parking lot or a library system), and write you
- Advanced: practise system design for common products (a chat system, a news feed, a URL shortener) and role-specific scenarios, an
- Interview day: prepare your environment, test your tools, and stay calm and communicative.
```

# 2. choosing_a_tech_domain.md

```
# How to Choose a Tech Domain or Role

## Role guide
"Tech" covers many different careers: building user interfaces, servers and APIs, mobile apps, data analysis, data science, machine

## Skill roadmap

### 1. Prerequisites and foundations
- Know yourself: what do you enjoy? Visible results on screen (frontend, mobile, design), solving logic and data problems (backend,
- Know your starting point: your education, current skills, domain knowledge, time available and how soon you need a job.
- Basics worth learning for almost any path: Git, the command line, SQL and one programming language.

### 2. Core and technical skill progression
- Level 1, compare similar roles. Data analyst answers business questions with SQL, Excel and dashboards. Data scientist adds statis
- Level 2, test before you commit: spend two to four weeks on a small project in each of two or three candidate paths (for example a
- Level 3, decide with evidence: shortlist one main path and one backup, check how many entry-level openings exist in your area, che

### 3. Practical tooling and ecosystem
- Compare roles: role guides, roadmaps and job postings (read the "requirements" sections).
- Try things cheaply: free tiers on cloud platforms, free notebooks, Figma's free plan, open datasets, beginner courses and hackatho
- Get feedback: mentors, communities and people in the role.

### 4. Hands-on projects and portfolio milestones
- Beginner: one small project in each of two or three directions.
- Intermediate: go deeper in the one you enjoyed, with a project that solves a real problem and is deployed or shared.
- Advanced: a portfolio of two or three strong projects in your chosen domain, matched to the skills in the job postings you want.
- Common mistakes: choosing only by salary or trends, learning five domains at once, and waiting for a perfect choice before startin
```

# 3. switching_into_tech.md

```
# How to Switch into a Tech Career from Another Field

## Role guide
People move into tech from teaching, sales, finance, operations, support, engineering, journalism, research and many other fields. Th

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: pick a target role and learn what it requires from role guides and job postings. Learn the fundamentals of your chos
- Programming and tools: for many paths, start with one language (often Python), SQL, Git and the command line. Some roles need litt
- Development environment: free learning platforms, a GitHub account and a small home lab or practice environment.
- Find your transferable skills: communication, analysis, project coordination, customer understanding and writing all count.

### 2. Core and technical skill progression
- Level 1, choose an adjacent role. Common bridges: accountants and finance staff to financial analysis, data analysis or SAP and ERI
- Level 2, build proof: complete two or three projects that match the target role, deploy or publish them, and write about what you l
- Level 3, enter the market: tailor your resume (see the resume note), highlight your earlier domain experience as an asset, use inte

### 3. Practical tooling and ecosystem
- Learning: structured courses, free documentation, project-based tutorials and practice labs.
- Proof: GitHub, portfolio sites, written case studies and deployed demos.
- Support: mentors, communities, study groups and people who made the same switch.

### 4. Hands-on projects and portfolio milestones
- Beginner: one small project that solves a problem from your old field (for example, an expense dashboard if you came from finance)
- Intermediate: two to three projects that match the role's postings, with a clear README and results.
- Advanced: an end-to-end project that combines your domain knowledge with the new skills, and a story that explains why the switch m
- Common mistakes: collecting certificates without building anything, learning too many tools at once, skipping the fundamentals, and
```

# 4. linux_administrator.md

```
# Linux Administrator

## Role guide
A Linux administrator installs, configures, maintains and secures Linux servers. The work covers users and permissions, processes and

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: the Linux file system hierarchy, how processes and services work, and networking basics (TCP/IP, DNS, DHCP, NTP).
- Programming: Bash shell scripting first, with Python as a useful addition.
- Development environment: a Linux virtual machine on your own computer, the command line, a text editor such as vim or nano, and Git
- Switching from another field: help desk and Windows administration are the usual routes. A computer science or IT degree is common,

### 2. Core and technical skill progression
- Level 1, fundamentals: Linux commands, file management, permissions, users and groups, process management, package management and t
- Level 2, core competencies (RHCSA level): services with systemd, partitions and file systems, logical volume management, network co
- Level 3, advanced: automation with Ansible (RHCE), system hardening, high availability, performance tuning, enterprise monitoring
- One 12-month learning path spreads the learning over phases: Linux basics, RHCSA preparation around months 3 and 4, advanced admin

### 3. Practical tooling and ecosystem
- Systems: Red Hat Enterprise Linux and compatible systems, Ubuntu, systemd, firewalld and iptables, SELinux.
- Storage: LVM, RAID, NFS.
- Automation: Bash, Python, Ansible.
- Monitoring: Nagios, Zabbix, Prometheus, Grafana.
- Virtualization and containers: KVM, Docker.

### 4. Hands-on projects and portfolio milestones
- Beginner: set up a Linux virtual machine, create users and permissions, and write a script that performs regular system checks and
- Intermediate: build a three-node Linux lab with logical volume storage, a shared file system over NFS, a firewall, SELinux in enfo
- Advanced: automate the whole lab setup with Ansible, add Prometheus and Grafana monitoring, write a hardening checklist and run fa
```

# 5. ux_researcher.md

```
# UX Researcher

## Role guide
A UX researcher studies user behavior, needs and pain points to improve a product and to help teams avoid building the wrong thing. T

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: research fundamentals (qualitative and quantitative), basic statistics, research ethics and participant consent, and
- Skills: active listening, empathy, communication and storytelling, plus persistence and problem solving.
- Development environment: survey tools, remote usability testing tools, spreadsheets, notes and recording tools.
- Switching from another field: transferable skills such as interviewing, synthesis and communication carry over, but you still need

### 2. Core and technical skill progression
- Level 1, fundamentals: user interviews and moderated usability testing first, because these are the most commonly required skills i
- Level 2, core competencies: survey design, synthesis (affinity mapping, themes), personas and journey maps, mixed-methods research,
- Level 3, advanced: research strategy that shapes the roadmap, continuous discovery with product teams, experiment and A/B test anal

### 3. Practical tooling and ecosystem
- Methods: interviews, surveys, usability studies, diary studies, card sorting, A/B tests, accessibility testing.
- Tools: remote testing tools (Maze, Lookback), survey tools, analytics tools, research repositories.
- Reading: books such as Just Enough Research and Interviewing Users are commonly recommended.

### 4. Hands-on projects and portfolio milestones
- Beginner: five user interviews about an everyday task, with a summary of themes.
- Intermediate: a usability test of a public website with five participants, a prioritized list of problems and a short report. Volun
- Advanced: a mixed-methods study (interviews plus a survey) on a real product, with recommendations presented to stakeholders.
```

# 6. network_engineer.md

```
# Network Engineer

## Role guide
A network engineer designs, implements, manages and troubleshoots computer networks: local networks (LAN), wide area networks (WAN),

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: the OSI and TCP/IP models, IP addressing and subnetting, Ethernet, and the role of devices such as routers, switches
- Programming: scripting in Python, Bash or PowerShell for automation, which matters more every year.
- Development environment: the command-line interface of network devices, a network simulator or lab tool for practice, and Wireshark
- Switching from another field: help desk, system administration and IT support are common starting points. Hands-on practice in a si

### 2. Core and technical skill progression
- Level 1, fundamentals: core protocols (TCP, UDP, IP, HTTP and HTTPS, DNS, DHCP, ARP, ICMP, SNMP), routing and switching basics (sta
- Level 2, core competencies: routing protocols (OSPF, EIGRP, BGP), network design (topologies, redundancy, scalability, reliability
- Level 3, advanced: network automation and programmability (Python, Ansible, APIs), zero trust designs, software-defined networking
- One plan spreads the learning over about a year: roughly 3 months of foundations, 5 months of design, security and wireless, then a

### 3. Practical tooling and ecosystem
- Network platforms: Cisco, Juniper, and firewalls from vendors such as Palo Alto and Fortinet.
- Monitoring and analysis: Wireshark, Nagios, Zabbix, SolarWinds.
- Automation: Python and Ansible.
- Cloud: virtual networks on AWS, Azure or Google Cloud.
- Practice tools: network simulators and emulators for building labs.

### 4. Hands-on projects and portfolio milestones
- Beginner: subnetting practice and a small office network in a simulator with VLANs and static routes.
- Intermediate: a multi-site network with OSPF, access control lists and a site-to-site VPN, with diagrams, and a Wireshark capture t
- Advanced: automate configuration backups and changes with Python or Ansible, and design hybrid connectivity between an office netwo
```

# 7. infrastructure_engineer.md

```
# Infrastructure Engineer

## Role guide
An infrastructure engineer designs, builds and operates the infrastructure that applications run on: physical and virtual servers, st

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: server and operating system basics, networking (TCP/IP, VLANs, routing, firewalls, VPNs, load balancers), storage co
- Programming: Bash, Python and PowerShell for automation.
- Development environment: a home lab with a hypervisor and several virtual machines, Git, and the command line on Linux and Windows
- Switching from another field: system administrators, network administrators and help desk staff with hands-on experience move into

### 2. Core and technical skill progression
- Level 1, fundamentals: Windows and Linux server administration, networking fundamentals, basic virtualization and basic monitoring
- Level 2, core competencies: virtualization platforms (VMware vSphere, Hyper-V, KVM or Proxmox), storage systems with backup and rep
- Level 3, advanced: designing for high availability and disaster recovery, capacity planning, performance tuning, hybrid cloud, cont

### 3. Practical tooling and ecosystem
- Virtualization: VMware vSphere, Hyper-V, KVM, Proxmox, Nutanix.
- Automation: Terraform, Ansible, CloudFormation, Pulumi, and scripting.
- Monitoring and logging: Prometheus, Grafana, Zabbix, Nagios, ELK.
- Storage and networking: SAN, NAS, Ceph, ZFS, enterprise switches and firewalls.
- Cloud: AWS, Azure, Google Cloud.

### 4. Hands-on projects and portfolio milestones
- Beginner: a virtualization lab with several virtual machines and basic networking.
- Intermediate: automate virtual machine provisioning with Ansible or Terraform, and build monitoring dashboards with alerts.
- Advanced: design a highly available infrastructure with backup and replication, test a disaster recovery drill, and document it wit
```

# 8. tech_resume_guide.md

```
# How to Write a Tech Resume (All Experience Levels)

## Role guide
A tech resume has one job: to get you the interview. Large companies often use Applicant Tracking Systems (ATS) that scan resumes fo

## Skill roadmap

### 1. Prerequisites and foundations
- Know the target role first. Read several job postings for it and note the recurring skills, tools and responsibilities. Some guides
- Collect your raw material: projects, internships, jobs, certifications, achievements and links (GitHub, portfolio, LinkedIn).
- Tools: a simple word processor, saved as PDF or DOCX as the employer asks. Avoid design tools such as Canva, graphics and charts, h

### 2. Core and technical skill progression
- Level 1, structure and format: keep it to one page if you are a student or early in your career (one guide says one page is correct
- Level 2, content that works: write every bullet as an action verb, what you did, and a measurable result (for example, "improved en
- Level 3, tailoring and polish: change your resume for each job by using the exact keywords from the posting naturally in your skil

### 3. Practical tooling and ecosystem
- Writing: a plain document editor and a PDF export.
- Checking: ATS resume checkers, a proofreader and a mentor review.
- Evidence: a GitHub profile with readable READMEs, a portfolio site for designers and developers, and optional certifications.

### 4. Hands-on projects and portfolio milestones
- Beginner: write a one-page resume with a skills section, two or three projects and education.
- Intermediate: rewrite every bullet in the action, task and result format, add numbers, and create a tailored version for one speci
- Advanced: keep a master resume of everything you have done and cut it down for each application. Pair it with a portfolio and a sho
- Common mistakes: one resume for all jobs, long paragraphs instead of bullets, padding the skills list, giving a non-tech job four h
```

# 9. sap_consultant_developer.md

```
# SAP Consultant / SAP Developer

## Role guide
SAP professionals work on SAP, the enterprise software many large companies run their finance, procurement, sales, manufacturing and

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: business processes in finance, procurement, sales and manufacturing, ERP ideas, databases and SQL. For the technical
- Programming: ABAP for the technical track. SQL and some JavaScript for Fiori and UI5 development.
- Development environment: SAP GUI, SAP training and learning systems (the SAP Learning Hub and SAP Learning Journeys), and an edito
- Switching from another field: accountants, supply chain and sales professionals often become functional consultants, and Java and

### 2. Core and technical skill progression
- Level 1, fundamentals: ERP concepts, one module's business process end to end (for example FI, MM or SD), or the ABAP basics (data
- Level 2, core competencies: object-oriented ABAP, BAPIs, BAdIs, user exits, the enhancement framework, IDocs and ALE, workflows an
- Level 3, advanced: the RESTful ABAP programming model (RAP), the clean core approach and side-by-side extensions on SAP Business Te

### 3. Practical tooling and ecosystem
- Core: SAP S/4HANA, SAP ECC, ABAP, SAP HANA.
- Front end and services: SAPUI5, Fiori Elements, OData, SAP Gateway.
- Quality and transport: ABAP Test Cockpit, ABAP Unit, change and transport management, Git.
- Process tools: SAP Signavio for process modeling.

### 4. Hands-on projects and portfolio milestones
- Beginner: an ALV report and a Smart Form, or walk through a standard purchasing or sales process in a training system.
- Intermediate: an OData service with a Fiori app, or a configured end-to-end order-to-cash scenario.
- Advanced: a RAP-based application with a Fiori front end, or a documented end-to-end implementation scenario including testing and
```

# 10. erp_consultant.md

```
# ERP Consultant

## Role guide
An ERP (Enterprise Resource Planning) consultant helps organizations choose, configure and run software that joins finance, human re

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: business processes in finance, procurement, inventory, sales, manufacturing and HR, process mapping and business pr
- Programming and data: SQL basics and strong spreadsheet skills. Technical consultants add scripting or a platform language.
- Development environment: a sandbox or trial of one ERP platform, process modeling tools, and documentation habits.
- Qualification: a degree in IT, business, finance or a related field is common, along with experience in business operations. Strong
- Switching from another field: accountants, supply chain professionals and business analysts are common entrants.

### 2. Core and technical skill progression
- Level 1, fundamentals: ERP concepts and one platform and module in depth, plus how to document a business process.
- Level 2, core competencies: gathering requirements, fit-gap analysis, configuration, data migration, test planning and user accepta
- Level 3, advanced: leading implementations through their phases (discovery, design, build, test, go-live and support), change manag

### 3. Practical tooling and ecosystem
- Platforms: SAP S/4HANA, Oracle Cloud Applications, Microsoft Dynamics 365, NetSuite, Workday, Infor, Odoo.
- Supporting tools: process modeling and workflow design tools, data migration tools, project and ticketing tools.
- Credentials: platform-specific certifications and industry certificates.

### 4. Hands-on projects and portfolio milestones
- Beginner: map an order-to-cash and a procure-to-pay process in diagrams.
- Intermediate: configure a demo company in a trial or open-source ERP and run a sample sales and purchase cycle.
- Advanced: write a mock implementation plan with a fit-gap analysis, a data migration approach, a test plan and training material.
```

# 11. performance_test_engineer.md

```
# Performance Test Engineer

## Role guide
A performance test engineer checks how an application behaves under real-world and peak load: how fast it responds, how many users it

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: testing fundamentals, how web applications and APIs work (HTTP, requests and responses), networking basics, operati
- Programming: scripting in Java, JavaScript, Groovy or Python depending on the tool, plus SQL.
- Development environment: JMeter, Postman, Git, and monitoring dashboards.
- Switching from another field: QA engineers, automation testers and developers move across most easily.

### 2. Core and technical skill progression
- Level 1, fundamentals: performance testing concepts, building JMeter test plans, parameterization and correlation, and reading basi
- Level 2, core competencies: modern tools (k6 with JavaScript, Gatling with its Scala or Java DSL, awareness of LoadRunner), worklo
- Level 3, advanced: distributed load generation (for example on Kubernetes or with cloud tools such as BlazeMeter), capacity planni

### 3. Practical tooling and ecosystem
- Load tools: Apache JMeter, k6, Gatling, LoadRunner, BlazeMeter, Artillery.
- Monitoring: Grafana, Prometheus, InfluxDB, APM tools.
- Pipelines: Jenkins, GitHub Actions.

### 4. Hands-on projects and portfolio milestones
- Beginner: a JMeter test for a sample web app with a simple report.
- Intermediate: a k6 script with thresholds, stages and a Grafana dashboard.
- Advanced: design a load test for a peak-traffic event such as a big sale on an e-commerce checkout, plus a soak test that hunts a m
```

# 12. qa_engineer.md

```
# QA Engineer

## Role guide
A QA (quality assurance) engineer makes sure software works as intended before and after release. The work includes understanding re

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: the software development life cycle (SDLC) and software testing life cycle (STLC), Agile and Scrum, testing types (
- Programming: SQL, because many QA roles need database queries to check data. A programming language such as Python, Java or JavaSc
- Development environment: Jira for bug tracking, TestRail or Excel for test cases, Postman for APIs, Git and GitHub.
- Soft skills: communication, teamwork, attention to detail and problem solving matter as much as tools.

### 2. Core and technical skill progression
- Level 1, fundamentals: writing test plans, scenarios and test cases, black-box design techniques (equivalence partitioning and bou
- Level 2, core competencies: API testing (HTTP methods, JSON validation) with Postman, SQL checks on data, test management, basic we
- Level 3, advanced: automation frameworks (Page Object Model, data-driven), CI/CD integration, performance basics with JMeter, secu
- Common mistakes: learning tools without the concepts behind them, and jumping to automation before manual testing is solid.

### 3. Practical tooling and ecosystem
- Tracking and management: Jira, TestRail, Bugzilla.
- API and data: Postman, SQL clients.
- Automation: Selenium, Playwright, Cypress, Appium for mobile, pytest, JUnit, TestNG.
- Pipelines: Jenkins, GitHub Actions.

### 4. Hands-on projects and portfolio milestones
- Beginner: write test cases for a real website or app, find bugs and document them, and publish the work on GitHub.
- Intermediate: a Postman collection for a public API with assertions, and a full test plan with test cases and a bug report set for
- Advanced: a complete test strategy for a sample application with manual, API and automated checks running in a pipeline.
```

# 13. enterprise_application_developer.md

```
# Enterprise Application Developer

## Role guide
An enterprise application developer designs, builds, integrates and maintains the large business applications that organizations run

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: object-oriented design, databases and SQL, web services and HTTP, XML and JSON, and enterprise architecture ideas s
- Programming: Java (with Spring Boot) or C# and .NET.
- Development environment: Git, an IDE, Maven or NuGet, Linux basics, and a CI tool.
- Switching from another field: developers from web, mobile or testing backgrounds can move in. Knowledge of the business domain, su

### 2. Core and technical skill progression
- Level 1, fundamentals: Java or .NET fundamentals, a web framework, REST APIs, SQL, and unit testing. A common learning project is a
- Level 2, core competencies: SOAP and REST web services, enterprise integration patterns, messaging with IBM MQ, Kafka or RabbitMQ,
- Level 3, advanced: integration platforms (Apache Camel, Red Hat Fuse, IBM App Connect, MuleSoft), designing for high availability a

### 3. Practical tooling and ecosystem
- Languages and frameworks: Java, Spring Boot, .NET.
- Integration and messaging: IBM MQ, Apache Kafka, RabbitMQ, Apache Camel, MuleSoft, ESB platforms.
- Servers and databases: Tomcat, WebLogic, JBoss, SQL Server, Oracle.
- Workflow and low-code: BPMN workflow platforms and low-code tools in some organizations.

### 4. Hands-on projects and portfolio milestones
- Beginner: a banking or inventory application with a database and REST API.
- Intermediate: a service that exposes both REST and SOAP interfaces, and an integration between two systems through a message queue
- Advanced: an integration flow built with Apache Camel or a similar tool, with error handling, monitoring and a short architecture w
```

# 14. salesforce_developer.md

```
# Salesforce Developer

## Role guide
A Salesforce developer customizes and extends the Salesforce CRM platform so a company can manage customers, sales and service in the

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: CRM ideas (leads, accounts, contacts, opportunities, cases), relational data modeling, and the Salesforce platform a
- Programming: object-oriented programming (Apex is similar to Java), SOQL, and web basics (HTML, CSS and JavaScript for LWC).
- Development environment: a free Salesforce Developer Edition org, Trailhead for guided learning, VS Code with Salesforce extensions
- Switching from another field: administrators add coding, Java and JavaScript developers add platform knowledge, and business analys

### 2. Core and technical skill progression
- Level 1, fundamentals: Salesforce Administrator skills (objects, fields, page layouts, security, reports and dashboards, automatio
- Level 2, core competencies: Apex classes and triggers, SOQL and SOSL, DML operations, governor limits and bulkification, test class
- Level 3, advanced: asynchronous Apex (batch and queueable), integrations with REST and SOAP callouts, event-driven design with Plat

### 3. Practical tooling and ecosystem
- Platform tools: Apex, SOQL, Lightning Web Components, Flow, Visualforce.
- Development: VS Code, Salesforce CLI, scratch orgs, Git.
- Learning and certification: Trailhead, Administrator, Platform App Builder, Platform Developer I and II.

### 4. Hands-on projects and portfolio milestones
- Beginner: custom objects, fields and a Flow that automates a business rule.
- Intermediate: an Apex trigger with a test class that follows bulkification rules, and a Lightning Web Component on a record page.
- Advanced: a complete app that combines admin and developer skills, with an integration to an outside API, and deployed with source
```

# 15. iot_developer.md

```
# IoT Developer

## Role guide
An IoT (Internet of Things) developer builds connected devices and the systems around them from end to end: sensors and microcontroll

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: basic electronics (sensors, actuators, power), networking (Wi-Fi, TCP/IP), how cloud services work, and IoT archite
- Programming: C or C++ for microcontrollers, Python or MicroPython for quick prototypes and cloud scripts, and basic JavaScript for
- Development environment: Arduino IDE, PlatformIO in VS Code, or ESP-IDF, Git, and a few development boards such as an ESP32 and a E
- Switching from another field: electronics students and software developers both move in easily, since each side learns the other's

### 2. Core and technical skill progression
- Level 1, fundamentals: programming a microcontroller (Arduino, NodeMCU or ESP32), GPIO, reading sensors and controlling actuators,
- Level 2, core competencies: IoT protocols (MQTT, HTTP and REST, CoAP, Bluetooth Low Energy, LoRa), running an MQTT broker such as M
- Level 3, advanced: IoT security (TLS encryption, protected credentials, secure over-the-air updates), edge, fog and cloud computing

### 3. Practical tooling and ecosystem
- Hardware: ESP32, NodeMCU, Arduino, Raspberry Pi, common sensors.
- Software: MicroPython, Arduino (C++), ESP-IDF, PlatformIO.
- Messaging and cloud: MQTT with Mosquitto, AWS IoT Core, Azure IoT, ThingsBoard, Node-RED.
- Data and dashboards: InfluxDB, Grafana, Docker.

### 4. Hands-on projects and portfolio milestones
- Beginner: read a temperature and humidity sensor and print it over serial, control a relay from a button.
- Intermediate: an IoT weather station that publishes over MQTT to a cloud dashboard, and a home automation project controlled from a
- Advanced: an end-to-end product with a mobile app, a secured cloud pipeline with encryption and over-the-air updates, and monitorir
```

# 16. systems_software_engineer.md

```
# Systems Software Engineer

## Role guide
A systems software engineer develops low-level software that interacts directly with hardware or system infrastructure: operating sys

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: operating system internals (processes, threads, scheduling, virtual memory, interrupts, I/O, synchronization), compu
- Programming: strong C, then C++. Python for scripting and tests.
- Development environment: Linux, GDB, Valgrind, perf, Git, build systems (Make, CMake), and QEMU or virtual machines for safe experi
- Switching from another field: embedded engineers, backend developers and security engineers move across by deepening their OS and (

### 2. Core and technical skill progression
- Level 1, fundamentals: C proficiency, data structures, Linux system programming (system calls, processes, signals, file I/O), and c
- Level 2, core competencies: concurrency and multithreading, memory management, networking with sockets, Linux kernel modules and s
- Level 3, advanced: kernel internals, drivers for several classes of devices (bus, block, wireless, display, USB), new hardware brir

### 3. Practical tooling and ecosystem
- Languages: C, C++.
- Platforms: Linux kernel, and sometimes Windows or macOS kernel frameworks.
- Debug and performance tools: GDB, Valgrind, perf, kernel debuggers, WinDbg on Windows.
- Hardware interfaces: PCIe, SPI, I2C, USB, DMA.

### 4. Hands-on projects and portfolio milestones
- Beginner: a small shell, a thread pool or a simple memory allocator in C.
- Intermediate: a Linux kernel module and a character device driver.
- Advanced: a driver for a real peripheral on a development board, with documentation, and a patch contributed to an open-source proj
```

# 17. sdet.md

```
# SDET (Software Development Engineer in Test)

## Role guide
An SDET is a software engineer who specializes in quality. Instead of only running or scripting tests, an SDET designs and builds the

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: object-oriented programming, data structures and algorithms basics, design patterns (such as the Page Object Model),
- Programming: strong skills in one language used in test code (Java, C#, TypeScript or Python), plus SQL.
- Development environment: Git, Linux, an IDE, Docker basics, and a CI tool.
- Switching from another field: developers learn test strategy, and manual or automation testers deepen their software engineering s

### 2. Core and technical skill progression
- Level 1, fundamentals: manual testing foundations, coding practice, first UI and API automation with Selenium or Playwright, and w
- Level 2, core competencies: framework design (page objects, data-driven, hybrid, BDD), API tests written in code for REST and Grap
- Level 3, advanced: designing frameworks that stay maintainable and reusable, quality gates and parallel execution in pipelines, co

### 3. Practical tooling and ecosystem
- Test tools: Playwright, Selenium, Cypress, Appium, REST Assured, Karate.
- Languages: Java, C#, TypeScript, Python.
- Delivery: Jenkins, GitLab CI, GitHub Actions, Azure DevOps, Docker.
- Performance and API extras: JMeter or k6, Postman.

### 4. Hands-on projects and portfolio milestones
- Beginner: automated UI and API tests for a demo app with clean structure.
- Intermediate: a test framework with reports, test data handling and a CI pipeline, plus contract tests between two services.
- Advanced: a containerized test platform with parallel execution, quality gates and a results dashboard, with documentation explain
```

# 18. systems_administrator.md

```
# Systems Administrator

## Role guide
A systems administrator installs, configures, maintains and secures an organization's servers and user systems, usually both Windows

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how operating systems work (Windows Server and Linux), networking basics (TCP/IP, DNS, DHCP, time synchronization),
- Programming: PowerShell for Windows and Bash or Python for Linux and cross-platform automation.
- Development environment: a home lab with virtual machines, the command line on both systems, and documentation habits.
- Switching from another field: IT support and help desk are the most common starting points, and certifications such as CompTIA A+ a

### 2. Core and technical skill progression
- Level 1, fundamentals: IT support basics, Windows and Linux basics, user and group management, file permissions, and basic trouble
- Level 2, core competencies: Active Directory and Group Policy, DNS and DHCP servers, file and print services, patch management, ba
- Level 3, advanced: automation with Ansible, PowerShell and Terraform, hybrid and cloud administration (AWS or Azure), disaster rec

### 3. Practical tooling and ecosystem
- Operating systems and directory: Windows Server, Active Directory, Group Policy, Linux distributions.
- Virtualization: VMware vSphere, Hyper-V.
- Automation: PowerShell, Bash, Ansible.
- Monitoring: Nagios, Zabbix.
- Cloud and identity: Microsoft 365, Azure, AWS.

### 4. Hands-on projects and portfolio milestones
- Beginner: build a small domain in a home lab with a domain controller, users and group policies.
- Intermediate: automate user creation with PowerShell, and set up backups and test a restore.
- Advanced: build a highly available service with monitoring, automate patching, and write a disaster recovery runbook.
```

# 19. firmware_engineer.md

```
# Firmware Engineer

## Role guide
A firmware engineer develops, tests and maintains the low-level code that runs directly on a device's hardware. Responsibilities inc

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: digital and basic analog electronics, microcontroller architecture (ARM Cortex-M is most common), memory and registe
- Programming: Embedded C is essential, with C++ and assembly knowledge useful. Python for tooling and tests.
- Development environment: Keil, IAR, MPLAB, STM32CubeIDE, ESP-IDF or PlatformIO, Git, a debugger probe, and the ability to read data
- Switching from another field: electronics graduates are common, and software developers can enter by building hardware projects.

### 2. Core and technical skill progression
- Level 1, fundamentals (junior): Embedded C, UART, SPI, I2C, USB and CAN basics, simple firmware features and defect fixing. A first
- Level 2, core competencies (intermediate): an RTOS such as FreeRTOS, writing your own hardware abstraction layer for GPIO, UART and
- Level 3, advanced: bootloaders over UART, USB and over the air, secure firmware, firmware architecture design, optimization for mem

### 3. Practical tooling and ecosystem
- Languages: Embedded C, C++, assembly.
- IDEs and tools: Keil, IAR, MPLAB, STM32CubeIDE, ESP-IDF, PlatformIO, debuggers and emulators.
- Operating systems: FreeRTOS, Zephyr, bare-metal approaches.
- Protocols: UART, SPI, I2C, CAN, USB, BLE, TCP/IP, MQTT.

### 4. Hands-on projects and portfolio milestones
- Beginner: a UART echo, then a temperature sensor on I2C with an LCD.
- Intermediate: an RTOS-based smart sensor node and your own mini hardware abstraction layer.
- Advanced: a bootloader with an over-the-air firmware updater and a secure update check, and contributions to an open-source firmwa
```

# 20. ux_designer.md

```
# UX Designer

## Role guide
A UX (user experience) designer takes care of the end-to-end experience of a product, from the first screen to finishing a task. The

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: design thinking, usability principles and the Laws of UX, basic psychology of how people perceive and decide, acces
- Tools: Figma for wireframes and prototypes, plus tools for usability testing such as Maze or Lookback.
- Development environment: whiteboards and sketching, collaboration tools, and basic knowledge of what developers can build.
- Switching from another field: people from graphic design, front-end development, research and product management move in. Prior UI

### 2. Core and technical skill progression
- Level 1, fundamentals: UX basics, personas, user journeys, information architecture, wireframing in Figma, and simple usability tes
- Level 2, core competencies: completing an end-to-end case study from research to prototype, interaction design, accessible design,
- Level 3, advanced: design systems, UX strategy, mixed-methods research, measuring outcomes (task success, time on task, satisfacti
- One roadmap suggests roughly 6 to 9 months of part-time study to reach a job-ready portfolio, with a job search stretching toward t

### 3. Practical tooling and ecosystem
- Design and prototyping: Figma, Framer, Adobe XD, Sketch.
- Research and testing: Maze, Lookback, survey tools, analytics.
- Collaboration: FigJam, Miro, Jira.

### 4. Hands-on projects and portfolio milestones
- Beginner: sketch and wireframe a simple app idea, such as a coffee shop ordering app, and test it on five people.
- Intermediate: an end-to-end UX case study with research, flows, wireframes, a clickable prototype and test findings.
- Advanced: three polished case studies, one of them a redesign of a real product with a measurable goal, and a design critique you l
```

# 21. embedded_software_engineer.md

```
# Embedded Software Engineer

## Role guide
An embedded software engineer designs, develops and tests the software that controls the functions of electronic devices, from smartv

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: digital electronics and basic circuits, how microcontrollers and microprocessors work (ARM, AVR, PIC, 8051), memory
- Programming: C first, then C++. Python for scripting, testing and automation.
- Development environment: development boards (Arduino, STM32, ESP32), IDEs such as Keil, IAR, STM32CubeIDE or PlatformIO, Git, and k
- Switching from another field: electronics and computer engineers are the usual route, and software developers can enter by learning

### 2. Core and technical skill progression
- Level 1, fundamentals: GPIO, timers, interrupts, ADC and PWM, serial communication with UART, SPI and I2C, and simple projects such
- Level 2, core competencies: a real-time operating system (FreeRTOS or Zephyr), task scheduling and synchronization, writing drivers
- Level 3, advanced: bootloaders and over-the-air updates, low-power design, security, firmware architecture, automated and hardware-

### 3. Practical tooling and ecosystem
- Languages and boards: C, C++, ARM Cortex-M boards, STM32, ESP32, Arduino.
- RTOS and libraries: FreeRTOS, Zephyr, vendor HAL libraries.
- Debug and build: JTAG or SWD debuggers, oscilloscopes, logic analyzers, CMake or Make, Git.
- Protocols: UART, SPI, I2C, CAN, plus Bluetooth Low Energy, Wi-Fi and Ethernet later.

### 4. Hands-on projects and portfolio milestones
- Beginner: LED and button control, a simple UART echo with a PC.
- Intermediate: a temperature sensor over I2C with an LCD display, and an RTOS-based smart sensor node.
- Advanced: your own hardware abstraction layer, a secure bootloader or an over-the-air firmware updater, and a documented, tested de
```

# 22. automation_test_engineer.md

```
# Automation Test Engineer

## Role guide
An automation test engineer writes code that tests software automatically, so checks run quickly and repeatedly instead of by hand. T

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: manual testing foundations (test design, defect life cycle), object-oriented programming, how web pages work (HTML,
- Programming: one language well. Java, Python and JavaScript or TypeScript are the common choices.
- Development environment: Git and GitHub, an IDE, the command line, Postman.
- Switching from another field: manual testers and developers both move into this role. The manual tester adds coding, and the develo

### 2. Core and technical skill progression
- Level 1, fundamentals: programming basics and OOP, locators and browser automation with Selenium or Playwright, running a first end
- Level 2, core competencies: test frameworks (TestNG, JUnit or pytest), the Page Object Model, data-driven, hybrid and BDD framework
- Level 3, advanced: parallel and cross-browser execution, Docker containers for test runs, reducing flaky tests, mobile automation w

### 3. Practical tooling and ecosystem
- UI automation: Selenium, Playwright, Cypress, Appium.
- Frameworks and runners: TestNG, JUnit, pytest, Cucumber.
- API: REST Assured, Postman, Karate.
- CI/CD and reporting: Jenkins, GitHub Actions, Azure DevOps, HTML report tools and dashboards.

### 4. Hands-on projects and portfolio milestones
- Beginner: automate a login and a few user flows on a demo site.
- Intermediate: a small framework from scratch with the Page Object Model, data-driven tests and reports, pushed to GitHub, and an AI
- Advanced: a framework in a Docker container with parallel runs and a CI pipeline that runs on every commit and publishes a report.
```

# 23. ui_designer.md

```
# UI Designer

## Role guide
A UI (user interface) designer designs how a product looks: layouts, typography, colors, spacing, icons, buttons and the other compor

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: visual design fundamentals (typography, color theory, spacing, contrast, hierarchy), Gestalt principles such as pro
- Tools: Figma is the industry standard for interface design, prototyping, collaboration and developer handoff. Adobe tools and Sket
- Development environment: basic HTML and CSS knowledge helps you design what developers can build.
- Switching from another field: graphic designers, front-end developers and artists move across by learning interface patterns and F

### 2. Core and technical skill progression
- Level 1, fundamentals: visual principles, Figma basics, and redesigning a simple mobile app screen using those principles. Good UI
- Level 2, core competencies: components and auto layout, Figma styles and variants, responsive and mobile-first design, high-fideli
- Level 3, advanced: scalable design systems and tokens, motion and interaction design, accessibility to WCAG standards, production-
- One guide estimates that part-time study for about 6 to 9 months can produce a job-ready portfolio. Treat this as a rough estimate

### 3. Practical tooling and ecosystem
- Design: Figma (with FigJam), Adobe XD or Illustrator, Sketch.
- Prototyping and motion: Figma prototyping, Framer, motion tools.
- Handoff and documentation: Figma Dev Mode, design system documentation.
- Learning: books on design principles and the free resources from Figma.

### 4. Hands-on projects and portfolio milestones
- Beginner: redesign one screen of an existing app and explain what you changed and why.
- Intermediate: a complete landing page and a small multi-screen mobile app with a consistent style.
- Advanced: a design system with documented components and rules, and a polished product UI kit with motion. Present each as a case s
```

# 24. manual_test_engineer.md

```
# Manual Test Engineer

## Role guide
A manual test engineer checks software by using it the way a real user would, without automation tools. The process generally runs f

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: computer basics, SDLC, STLC, Waterfall and Agile (including Scrum), testing types, black-box and white-box testing,
- Programming: none required to begin. SQL is a strong plus, and basic HTML and CSS help you understand web pages.
- Development environment: Jira or similar for bugs, TestRail or spreadsheets for test cases, and a browser with developer tools.
- Switching from another field: people with domain knowledge in banking, healthcare or e-commerce have a real advantage.

### 2. Core and technical skill progression
- Level 1, fundamentals: reading requirements, writing test scenarios and test cases, test plans, the bug life cycle, and writing cle
- Level 2, core competencies: exploratory testing beyond the script, regression planning (and when to use smoke or sanity testing), A
- Level 3, advanced: risk-based testing, test strategy, mentoring newer testers, domain specialization, and starting automation (bas

### 3. Practical tooling and ecosystem
- Test and defect management: Jira, TestRail, Bugzilla, spreadsheets.
- API and data: Postman, SQL clients.
- Device and browser checks: browser developer tools, cloud device and browser testing platforms.
- Certification often named: ISTQB Foundation.

### 4. Hands-on projects and portfolio milestones
- Beginner: write test cases for a familiar app such as a shopping or messaging app.
- Intermediate: test a demo e-commerce site end to end, log defects in a tracker, and write a test summary report.
- Advanced: a full test cycle for a sample project (plan, cases, execution, defect triage, regression pack) with a short document exp
```

# 25. robotics_software_engineer.md

```
# Robotics Software Engineer

## Role guide
A robotics software engineer writes the software that lets robots sense, decide and act. The areas include perception (cameras, LiDAR

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: linear algebra, calculus and probability, kinematics and dynamics basics, coordinate frames and transforms, data st
- Programming: C++ and Python.
- Development environment: Ubuntu Linux, ROS or ROS 2, Git, Docker, and simulators such as Gazebo or Isaac Sim.
- Switching from another field: mechatronics, electronics, computer science and mechanical engineers all enter this field. A degree

### 2. Core and technical skill progression
- Level 1, fundamentals: programming plus ROS 2 basics (nodes, topics, services, actions, TF), robot descriptions (URDF) and running
- Level 2, core competencies: perception with OpenCV and point clouds, sensor fusion and state estimation (EKF and UKF), SLAM and lo
- Level 3, advanced: manipulation, real-time performance on edge computers, machine-learning perception with PyTorch or TensorFlow, s

### 3. Practical tooling and ecosystem
- Frameworks: ROS 2, Nav2, MoveIt, Eigen, OpenCV.
- Simulation: Gazebo, Isaac Sim.
- Sensors: cameras, LiDAR, IMU, wheel encoders, GNSS.
- Tools: Docker, Git, CI, logging and telemetry tools.

### 4. Hands-on projects and portfolio milestones
- Beginner: a simulated differential-drive robot driven from the keyboard, with a ROS 2 node that reads a sensor.
- Intermediate: autonomous navigation in simulation with Nav2 and SLAM, and an object detection node.
- Advanced: a manipulator pick-and-place with MoveIt, and deployment on a real robot with telemetry and a documented sim-to-real comp
```

# 26. product_designer.md

```
# Product Designer

## Role guide
A product designer owns the design of a product from the first idea to the shipped feature and beyond. The role joins user experience

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: UX principles, visual design fundamentals, accessibility, and product thinking (user needs, business goals, metrics
- Tools: Figma for design, prototyping and handoff, plus a basic understanding of analytics.
- Development environment: a general understanding of how front-end code is built, so you can design what is feasible, and collaborat
- Switching from another field: UX designers add visual and product skills, and UI designers add research and product thinking. Prod

### 2. Core and technical skill progression
- Level 1, fundamentals: UX and UI foundations, wireframes, mockups and a first case study.
- Level 2, core competencies: product discovery and problem framing, user flows, building and using a design system, prototypes for t
- Level 3, advanced: product strategy and influencing the roadmap, systems thinking across a whole product, experimentation, design l

### 3. Practical tooling and ecosystem
- Design: Figma, FigJam, prototyping tools such as Framer.
- Product and data: analytics tools, experimentation platforms, Jira and product documentation tools.
- Research support: usability testing tools and survey tools.

### 4. Hands-on projects and portfolio milestones
- Beginner: redesign a sign-up or onboarding flow and state the metric you expect it to improve.
- Intermediate: an end-to-end product case study covering the problem, research, design, prototype and a testing result.
- Advanced: a case study with a measured outcome, a design system you built, and the experiment you designed to check whether it wor
```

# 27. network_administrator.md

```
# Network Administrator

## Role guide
A network administrator keeps an organization's network running day to day. The work includes configuring and maintaining routers, s

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: networking fundamentals (OSI and TCP/IP, IP addressing, subnetting), how Windows and Linux connect to a network, an
- Programming: light scripting in PowerShell or Bash to automate repeated tasks.
- Development environment: a home or simulator lab, SSH or terminal tools for connecting to devices, and a habit of documenting what
- Switching from another field: IT support and help desk staff are the most common entrants.

### 2. Core and technical skill progression
- Level 1, fundamentals: cabling, network devices, wireless basics, DHCP, DNS, switching, and basic troubleshooting with ping, trace
- Level 2, core competencies: configuring routers and switches from the command line, VLANs, access control lists, firewalls and VPN
- Level 3, advanced: automating routine tasks, hardening network security, capacity planning, working with vendors, and moving towar

### 3. Practical tooling and ecosystem
- Devices and vendors: Cisco, Juniper, Fortinet, Palo Alto.
- Monitoring: Wireshark, Nagios, Zabbix, SolarWinds.
- Services: DHCP and DNS servers, VPN gateways, wireless controllers.
- Workflow: ticketing systems and network diagrams.

### 4. Hands-on projects and portfolio milestones
- Beginner: a small home or office network in a simulator.
- Intermediate: configure a small business network with VLANs, DHCP, DNS and guest Wi-Fi in a simulator, and document it.
- Advanced: set up monitoring with alerts, automate configuration backups, and write an incident report for a simulated outage with
```